Anomaly Detection using Negative Selection Algorithms

Report on Assignment 2

Group 28

February 23, 2026

1 Introduction

Anomaly detection in system call sequences is a foundational approach to intrusion detection. Because Unix processes communicate with the kernel through system calls, these sequences can provide critical insight into the health of a system. The core premise is that legitimate processes (such as `sendmail`) generate predictable patterns during normal behavior. In contrast, anomalous system call sequences, such as a normally silent process suddenly opening hundreds of files, can be a strong indicator of a malicious exploit.

To tackle this, the Negative Selection Algorithm (NSA) can be used to classify sequences as either normal or anomalous. The NSA is a technique heavily inspired by the biological immune system. In nature, T-cells that react to the body's own cells (the "self") are eliminated in the thymus during development; only those that tolerate the self survive to recognize and attack foreign, non-self material. The algorithm computationally mirrors this biological process as it generates a repertoire of detectors that explicitly do not match any strings within a labeled "self" training corpus. At test time, any input sequence that is matched by at least one detector in the repertoire is flagged as an anomaly.

The objective of this project is to train a classifier using normal system call sequences to accurately detect anomalies in test datasets. We will be working with historical benchmark data originally collected by Stephanie Forrest's group at the University of New Mexico. Given a training corpus of normal system call sequences, the goal is to train a classifier that assigns an anomaly score to each sequence in the test sets ensuring that anomalous sequences score higher than normal ones. The test sets contain both normal and anomalous behavior. Ultimately, the quality of our classification and the algorithm's discriminative power will be evaluated using Receiver Operating Characteristic (ROC) visualizations and the Area Under the Curve (AUC) metric.

2 Methodology

2.1 Dataset

For this analysis, we utilized two primary datasets, `snd-cert` and `snd-unm`, located within the provided syscalls directory. These datasets capture the sequences of system calls (such as `open`, `read`, or `write`) made by Unix processes communicating with the kernel. Each dataset is structured into several specific files to facilitate training and evaluation:

- **Training Data** (`.train`): A single file containing sequences recorded exclusively during the normal, healthy behavior of a process.
- **Alphabet** (`.alpha`): A file containing the discrete alphabet used to encode the different system calls.
- **Test Data** (`.test`): Three separate test splits containing a mixture of both normal and anomalous system call sequences to evaluate the classifier.
- **Labels** (`.labels`): Corresponding files that provide the ground truth for the test sequences, using a binary classification system where 0 indicates a normal sequence and 1 signifies an anomalous one.

A defining characteristic of these datasets is that the recorded system call sequences are of variable lengths. Because the negative selection algorithm is designed to process fixed-length strings, this variable-length nature introduces a critical preprocessing step. Before training or testing can begin, the sequences must be split into fixed-length chunks.

2.2 Negsel2

To perform the negative selection, we utilized the provided Java-based implementation, `negsel2.jar`. This tool operates on the principle of r-contiguous detectors. By providing a "self" training file, a specified chunk size n, and a matching threshold r, the algorithm generates a repertoire of detectors of length n. During the testing phase, the tool evaluates each input string against this trained repertoire. It calculates an anomaly score based on the number of detectors that match the input. We utilize specific command-line switches, such as the `-c` flag which instructs the tool to count the matching patterns, and the `-l` flag which converts this raw count x into a more manageable logarithmic format using the formula $log_2(1+x)$. Consequently, a higher score indicates a greater deviation from the normal training data, signaling a higher degree of "non-self-ness." The tool is invoked via the command line using the following structure, where the `-alphabet` switch ensures the basis alphabet matches the unique system calls found in our dataset:
`java -jar negsel2.jar -alphabet file://<alpha> -self <train> -n <n> -r <r> -c -l < <input>`

2.3 Data Preprocessing and Chunking Strategy

Because the sequences stored in the intrusion detection files are no longer of a fixed length, the raw data cannot be fed directly into the algorithm. The `negsel2` tool requires fixed-length strings on its standard input, meaning we must pre-process each sequence into a set of fixed-length chunks of size n. To handle this, a two-step preprocessing and scoring strategy was developed.

2.3.1 Baseline

As an initial approach, the training file was passed as-is to `negsel2` via the `-self` flag, and chunking was applied exclusively to the test sequences. For classification, we split the sequences into chunks. Specifically, each test sequence was divided into non-overlapping substrings of length n using the following Python function:

```
1 def chunk_sequence(seq: str, n: int) -> list[str]:
2     return [seq[i:i+n] for i in range(0, len(seq) - n + 1, n)]
```

Each chunk was scored individually by the negative selection algorithm. To merge these counts, we averaged the individual chunk scores to produce a single composite anomaly score per test sequence. Mathematically, this is represented as:
$score(seq) = mean\{negsel\_score(chunk) \mid chunk \in chunks\_n(seq)\}$

While this approach yielded reasonable results on the `snd-cert` test split 1 (achieving an AUC of 0.9374 at $n = 15$, $r = 5$), a critical practical limitation became apparent. For the parameter set $n = 15$ and $r = 6$, runtimes exceeded one hour, making a thorough parameter sweep infeasible. The assignment brief explicitly warns that if running the algorithm on one set of parameters takes too long, you might run into time trouble. Therefore a more effective use of the data was required.

2.3.2 Chunked Training File for Optimization

To address the severe runtime bottleneck, the pre-processing logic was extended to the training data. The training sequences were also pre-processed into a set of fixed-length, non-overlapping chunks of length n. To optimize the process further, only the unique set of these chunks was written to a temporary file and passed to `negsel2` as the `-self` corpus.

```
1 def write_chunked_train(train_seqs: list[str], n: int) -> Path:
2     chunks = set()
3     for seq in train_seqs:
4         chunks.update(chunk_sequence(seq, n))
```

```
    tmp.write("\n".join(chunks) + "\n")
    return tmp
```

This drastically reduced the size of the "self" set presented to the algorithm. This smaller self set translates to fewer detectors needing to be matched against, which substantially lowered the runtime. Crucially, this refined approach also produced noticeably higher AUC scores on snd-unm (detailed in Table ...), suggesting that an explicitly compact and distinct set of normal patterns is highly effective at discriminating normal sequences from anomalous ones in this dataset.

2.3.3 Parameters

To tune the negative selection algorithm, two hyperparameters must be chosen: the chunk size n and the contiguous matching threshold r. To find the optimal balance for intrusion detection, we designed a grid search over the following parameter space:

- $n \in 7, 10, 15$
- $r \in 2, 3, 4, 5, 6$

Performance was initially estimated and tuned on test split 1 for both datasets, and subsequently validated on test splits 2 and 3 to ensure the classifier's generalizability.

3 Results

In this section, we evaluate the performance of the negative selection algorithm using the Area Under the Curve (AUC) metric. The AUC quantifies the algorithm's ability to successfully discriminate between normal system call sequences and anomalous ones. We present the findings in two phases: the initial baseline approach (using unchunked training data) and the optimized approach (where both training and test data were preprocessed into fixed-length chunks).

3.1 Unchunked Training Sequences

For our baseline evaluation, the training file was fed into the `negsel2` tool without explicit chunking, while the test sequences were split into non-overlapping chunks. We used the parameters $n = 10$ and $r = 4$ as a foundational starting point. To determine the optimal n and r parameters, we conducted a grid search on the first test split of the `snd-unm` dataset, because it's smaller than the `snd-cert` dataset, thus it's faster to run and test.

We validated a stable parameter combination ($n = 10$, $r = 4$) across all three test splits for both `snd-cert` and `snd-unm`. The algorithm demonstrated solid baseline performance (Table 1), achieving AUC scores around 0.94 on `snd-cert` and ranging from 0.93 to 0.97 on `snd-unm`.

Table 1: Baseline results across both datasets (Unchunked Training, $n = 10$, $r = 4$)

Dataset	**Split 1**	**Split 2**	**Split 3**
`snd-cert`	0.9350	0.9456	0.9459
`snd-unm`	0.9344	0.9778	0.9789

Table 2: AUC Grid Search on `snd-unm` Split 1 (Unchunked Training Data)

Chunk Size (n)	$r = 2$	$r = 3$	$r = 4$	$r = 5$	$r = 6$
7	0.8584	0.9396	0.9396	0.9388	0.9384
10	0.9272	0.9344	0.9344	0.9344	0.9344
15	NaN	NaN	NaN	0.9724	0.9724

The results from the grid search on the `snd-unm` dataset, represented in Table 2, indicate that a larger chunk size generally improves discriminative power, provided r is scaled appropriately. However, when running the parameter set $n = 15$ and $r = 6$ on the `snd-cert` runtime exceeded one hour, which reveals some computational inefficiency.

Notably, for $n = 15$ with $r < 5$, the algorithm produces NaN scores. These results are likely caused by numerical instability in `negsel2`'s internal computation. We will discuss this further in the discussion section.

3.2 Chunked Training Sequences

To overcome the runtime bottlenecks warned about in the assignment brief and improve detection accuracy, we applied the chunking preprocessing step to the normal training sequences as well. Only the unique chunks were passed as the "self" corpus.

Using the same parameter set of $n = 10$ and $r = 4$ as a starting point, we evaluated the performance across both datasets. The results in table 3 show AUC scores remaining above 0.97 for both datasets.

Table 3: Results across both datasets (Chunked Training, $n = 10$, $r = 4$)

Dataset	**Split 1**	**Split 2**	**Split 3**
`snd-cert`	0.9728	0.9821	0.9839
`snd-unm`	0.9804	0.9821	0.9846

Table 4: AUC Grid Search on `snd-unm` Split 1 (Chunked Training Data)

Chunk Size (n)	$r = 2$	$r = 3$	$r = 4$	$r = 5$	$r = 6$
7	0.9300	0.9700	0.9700	0.9900	0.9900
10	0.9700	0.9804	0.9804	0.9804	0.9804
15	NaN	NaN	NaN	0.9900	0.9900

3.2.1 Optimized Parameters

We tried the two best configurations from the earlier results out in more detail:

Table 5: Optimized validation across both datasets (Chunked Training, $n = 7$, $r = 5$). This took 44:42min to run on the test machine (Processor: Intel i5-8365U; 16 GB RAM) including chunking.

Dataset	**Split 1**	**Split 2**	**Split 3**
`snd-cert`	0.9774	0.9841	0.9748
`snd-unm`	0.9900	0.9630	0.9854

Table 6: Optimized validation across both datasets (Chunked Training, $n = 15$, $r = 5$). This took 25:48min to run on the test machine (Processor: Intel i5-8365U; 16 GB RAM) including chunking.

Dataset	**Split 1**	**Split 2**	**Split 3**
`snd-cert`	0.9756	0.9832	0.9746
`snd-unm`	0.9900	0.9732	0.9760

4 Discussion

Overall, the negative selection algorithm receives very high AUC scores across a wide variety of parameter-configurations on the `Intrusion Detection for Unix Processes` dataset, to the point where we were at first unsure of our implementation. Apart from the generally promising results, the following observations warrant a deeper discussion:

4.1 Numerical Instability at $n = 15$ With Low r

As we could repeatedly see with the results, running the tests with $n = 15$ and $r < 5$ as parameters produces `NaN` values. `Negsel2` computes detector counts analytically using an inclusion-exclusion formula over large combinatorial terms. With $n = 15$ and small r, individual terms in this alternating sum reach magnitudes on the order of 10^{18}. Subtracting these nearly-equal large values causes catastrophic floating-point errors, producing negative counts (e.g., -7.42×10^{18}, observed when running it without the -l flag). Since the `-l` flag computes $\log_2(1 + x)$ of these counts, taking the logarithm of a negative number yields `NaN`. For $r \geq 5$, the r-gram space is large enough that the numbers remain numerically stable and we can get useable results.

4.2 Relationship Between Chunk Size and AUC

An unexpected finding from the parameter sweep is the relationship between chunk size n and AUC: $n = 7$ and $n = 15$ both achieve AUC = 0.9900 in `snd-unm` Split 1 at $r = 5$, while $n = 10$ plateaus at 0.9804. We believe that this can be explained by the competing mechanisms through which the algorithm manages to discriminate good data from fraudulent data with each chunk size:

Small chunks ($n = 7$) can provide granularity due to their size. Each 1000-character sequence leads to about 142 chunks, and the large number of chunks increases the chance of detecting local anomalies.

On the other hand, large chunks ($n = 15$) are more informative. Longer chunks carry more contextual information, making normal patterns distinctive from anomalous ones. The chunks themselves are likely informative enough to separate the two classes.

Lastly, medium chunks of $n = 10$ fall in between these two values and instead of leading to a sweet-spot by combining the strengths of small and large chunks as we would have expected, they seem to not do either thing as well. The comparatively bigger chunks when contrasted with $n = 7$ lead to fewer samples, while the comparatively smaller chunks when contrasted with $n = 15$ provide less information for distinguishing between normal and anomalous test chunks. The results show that it appears to be better to utilize small or large chunks than to use medium ones.

4.3 Computational Cost

For a sequence of length L chunked with size n and matching threshold r, there are two different possible quantities that can define how expensive the program is to run. Non-overlapping chunking produces $\lfloor L/n \rfloor$ chunks, and each chunk requires $(n - r + 1)$ contiguous-window evaluations. The total matching windows per test sequence is therefore:

$$W(n, r) = \left\lfloor \frac{L}{n} \right\rfloor \cdot (n - r + 1) \tag{1}$$

For $L = 1000$ and $r = 5$:

	$n = 7$	$n = 15$
Chunks per sequence $\lfloor L/n \rfloor$	142	66
Windows per chunk $(n - r + 1)$	3	11
Total windows W	426	726

These factors compete: $n = 7$ produces more chunks, while $n = 15$ requires more matching windows per sequence. In this experiment with the given dataset and on our test machine, $n = 7$ ran much slower than $n = 15$, suggesting that the fixed per-chunk overhead is more demanding than the per-window matching cost. To be accurate it must be said that total runtime is just an indication of computational cost, not an accurate measure of it, but the results in this case were quite clear.

4.4 The "Best" Parameters

Both $n = 7$ and $n = 15$ with $r = 5$ achieve equivalent AUC of 0.9900 on `snd-unm` Split 1 and perform comparably across all test splits. We therefore consider them equivalent in terms of results. However, as $n = 15$ ran in 25:48 compared to 44:42 for $n = 7$, we would prefer it for this usecase. Used in practice to catch fraudulent programs, it might also be useful to run two algorithms with both chunk-sizes simultaneously, as they observe the data at a different scale and might be able to detect different

kinds of intrusions that the other one would overlook. $n = 15$ with $r = 6$ also received the same score as $n = 15$ with $r = 5$, but takes longer to run, so we did not observe it further.

It is also notable that the best parameters for intrusion detection are different from the best parameters for language classification. For language classification we found $r = 3$ to be the best parameter (with the given $n = 10$), while for intrusion detection the best value of r we could identify is 5.

5 Conclusion

Appendices

Task 1: Using the Negative Selection Algorithm

This section contains the questions and answers from "Your task" on page 2 of the second assignment.

1. *Compute the area under the receiver operating characteristic curve (AUC [1][2]) to quantify how well the negative selection algorithm with parameters $n = 10$ and different values of r ($r = 1$ until $r = 9$) discriminates individual English strings from Tagalog strings by using the files english.train for training and english.test as well as tagalog.test for testing. Which value of r belongs to what AUC in figure 1?*

   Plot 1 has an r-value of 1, plot 2 has an r-value of 7, and plot 3 has an r-value of 4.

2. *How does the AUC change when you modify the parameter r? Specifically, what behaviour do you observe at $r = 1$ and $r = 9$ and how can you explain this behaviour? Which value of r leads to the best discrimination?*

   When increasing the r-value beyond 3, the AUC curve "flattens", making it slightly worse at discrimination. At r-values 1, 8 and 9, the AUC value is close to 0.5, making it almost no better than random guessing.
   The best performance is achieved by $r = 3$ with an AUC score of 0.8311.

3. *The folder 'lang' contains strings from 4 other languages. Which languages can be best discriminated from English using the negative selection algorithm, and for which is this most difficult?*

   The best AUC scores for all languages were found with an r-value of 3. The AUC scores for each language compared to English are listed below:

   - **Middle-english:** 0.5424.
   - **Plautdietsch:** 0.7747.
   - **Hiligaynon:** 0.8397.
   - **Xhosa:** 0.8893.

   We observe that Middle-english is the hardest language to discriminate from English. This is a logical conclusion as English is a direct descendant of Middle-english, and both languages share vocabulary and structural patterns. The best AUC value of 0.5424 is close to random guessing, meaning that the algorithm produces only a few matches and struggles to distinguish Middle-english from regular English.

   On the other hand, Xhosa, Hiligaynon, and Plautdietsch were much easier to discriminate. Xhosa is the easiest to discriminate, with an AUC score of 0.8893. This is followed by Hiligaynon with an AUC of 0.8397. These languages are linguistically distant from English as they use different consonants and combine characters differently. Plautdietsch is the hardest out of these three to distinguish, with an AUC score of 0.7747. This may be due to the fact that it is a dialect of German and shares some similarities with the English language.

4. *We train the repertoire on strings of a certain length, but we could in theory input strings of other lengths. Explain in your own words the issue with providing strings that are too long/short.*

   Using shorter strings makes it more difficult to find matches as the sliding windo would be much smaller. This would likely result in many zero or non-matches. Longer strings would have the opposite effect. With longer input strings, we would be able to find more matching patterns. However, the anomalous part of a long string might be overshadowed by the high number of normal matches, making it harder to classify accurately.